CMSC433
Midterm 2 practice

An editable problem workbook

Types first, followed by code from signatures, monadic execution, State, verification conditions, laziness, and a mixed set.

How to answer

Open `study-guide.tex` and the relevant folder's `lesson.tex` in your editor. Replace the text inside `\answer {...}` for written answers. For coding questions, replace the `undefined` lines inside `codeanswer` blocks with your Haskell implementation. All answers can be written in LaTeX; no separate Haskell file is required. Rebuild the PDF with `.\build .ps1`. The companion `Practice.hs` files are optional for GHCi checks. The PDF is a compiled reading copy, not a fillable PDF form.

Practice basis

Spring 2025 sample Q1–Q5 and local lectures. No official Fall 2026 scope, timing, resource rules, or weighting is assumed. The mixed set matches the sample's question styles, not a verified current exam format. No solutions are included. Reference signatures are at the end.

Prepared November 9, 2026. Sources and apparent source typos: `SOURCES.md`.

Contents

1 Type foundations 1

2 Contextual and higher-order types 3

3 Implement functions from signatures 5

4 Evaluate monadic programs 7

5 State execution and implementation 9

6 Verification conditions 11

7 Laziness and summary-producing recursion 13

8 Mixed review: five sample-style questions 15

9 Lecture extensions: coverage unconfirmed 20

A Study reference 22

Chapter 1

Type foundations

Source: Sample Q1, p. 1; Monads and GenericMonads lecture signatures.

Give the **most general type**, including constraints, or write **ill-typed** and identify the incompatible types. Do not use GHCi until you have written your prediction. In this set, show at least one intermediate type or type equation for every inference.

1.1. Read the notation

(a) Insert parentheses to show the grouping of `a -> b -> c`.

Answer: [Type here.]

(b) Insert parentheses to show the grouping of `f x y`.

Answer: [Type here.]

(c) For `(a -> b) -> [a] -> [b]`, list the inputs and final output.

Answer: [Type here.]

1.2. Complete a type derivation

Infer `\p -> (snd p, fst p)`. Fill every field before the final answer.

Answer: p :: [ ]; snd p :: [ ]; fst p :: [ ]; pair :: [ ]; full type :: [ ].

1.3. Infer or reject

(a) `\f x -> Just (f x)`

Answer: [Type and reasoning.]

(b) `\xs -> [x | x <- xs, x]`

Answer: [Type and reasoning.]

(c) `\xs -> [even x | x <- xs]`

Answer: [Type and reasoning.]

(d) `\x -> (x, x == x)`

Answer: [Type and reasoning.]

(e) `\f -> (f True, f 'q')`

Answer: [Type or first conflict.]

(f) `\x -> x x`

Answer: [Type or first conflict; write the required equation for x.]

1.4. Construct an expression from its type

Implement each function. Do not use `undefined`, `error`, or a nonterminating expression as your answer. For `describeMaybe`, use `<$>` without pattern matching or do notation.

```
swapPair :: (a,b) -> (b,a)
swapPair = undefined  -- YOUR CODE

applyTwice :: (a -> a) -> a -> a
applyTwice = undefined  -- YOUR CODE

describeMaybe :: Show a => Maybe a -> Maybe String
describeMaybe = undefined  -- YOUR CODE
```

1.5. Diagnose and repair

The expression `map Just (Just True)` is intended to wrap each element of a list of booleans. Explain the original error. Replace only the argument to `map Just` with a suitable nonempty list and give the repaired type.

Answer: [Error, repair, type.]

1.6. Changed independent inference

Infer `\k xs -> k (map Just xs)`. State the argument type required of `k` and explain whether `k` needs a Functor constraint.

Answer: [Type and reasoning.]

Checkpoint: Explain your type equations, not just the final signatures. Revisit this set if you lose list/pair/Maybe constructors or omit necessary constraints.

Chapter 2

Contextual and higher-order types

Source: Sample Q1, p. 1; Monads lecture, Functor/Applicative; State lecture.

For every part, give the most general type or identify a type error. All parts are independent. Reference signatures are provided in the appendix. Keep the outer context in your answer and show how the payload type changes.

2.1. Functor and Applicative

(a) `(,) <$> Just False`

Answer: [Type of (,) after its first argument; whole type.]

(b) `(++) <$> Just "ab"`

Answer: [Type and reasoning.]

(c) `(++) <$> Just "ab" <*> Just "cd"`

Answer: [Type and reasoning.]

(d) `Just not <*> Just (3 :: Int)`

Answer: [Type or conflict.]

(e) `\f mx -> f <$> mx`

Answer: [Most general type and constraint.]

(f) `\mf mx -> mf <*> mx`

Answer: [Most general type and constraint.]

(g) `\mx f -> mx >>= f`

Answer: [Most general type and constraint.]

2.2. State types

Use `get :: State s s`, `put :: s -> State s ()`, and `modify :: (s -> s) -> State s ()`. For each answer, explicitly separate the stored-state type from the returned-value type.

(a) `get >>= \xs -> put (reverse xs)`

Answer: [Stored type; returned type; whole type.]

(b) `modify (map not)`

Answer: [Stored type; returned type; whole type.]

(c) `put 'r' >> put False`

Answer: [Type or incompatible state requirements.]

(d) `(,) <$> get`

Answer: [Stored type; returned type; whole type.]

(e) `\f -> f <$> get`

Answer: [Type of f; whole type.]

(f) `get >>= \x -> return (x, x + 2)`

Answer: [Stored type; returned type; constraints; whole type.]

2.3. Implement from contextual signatures

Use `<$>` and/or `<*>`; no pattern matching or do notation. `choosePairs` must enumerate each left choice before advancing to the next.

```
applyAvailable :: Maybe (a -> b) -> Maybe a -> Maybe b
applyAvailable = undefined  -- YOUR CODE

pairAvailable :: Maybe a -> Maybe b -> Maybe (a,b)
pairAvailable = undefined  -- YOUR CODE

choosePairs :: [a] -> [b] -> [(a,b)]
choosePairs = undefined  -- YOUR CODE
```

2.4. Changed independent inference

Infer `\f -> modify f >> get`. Give the required relationship between the input and output types of `f`. Explain which action determines the returned type of the full computation.

Answer: [Most general type and reasoning.]

Chapter 3

Implement functions from signatures

Source: Sample Q2, p. 2 and Q4(b), p. 4; GenericMonads lecture.

Write code directly in these answer blocks. Implement the described behavior, not merely an expression that type-checks. No `undefined`, `error`, or intentional nontermination. The optional Haskell companion repeats these tasks.

3.1. Collect present values

Return all values in order when every element is `Just`; otherwise return `Nothing`. An empty list succeeds with an empty result. Use explicit recursion, not `sequence`, `traverse`, `mapM`, or equivalent helpers.

```
collectMaybe :: [Maybe a] -> Maybe [a]
collectMaybe = undefined  -- YOUR CODE
```

Answer: [Types of the head, recursive result, and constructed payload.]

3.2. Generalize the operation

Implement left-to-right traversal and action collection. Use explicit recursion and do/bind/return; do not call traversal, sequencing, or monadic fold helpers. For `gather`, do not delegate to `walkM`.

```
walkM :: Monad m => (a -> m b) -> [a] -> m [b]
walkM = undefined  -- YOUR CODE

gather :: Monad m => [m a] -> m [a]
gather = undefined  -- YOUR CODE
```

3.3. Rewrite using a library combinator

Reimplement `gather` with exactly one call to library `mapM`, with no explicit recursion. `mapM :: Monad m => (x -> m y) -> [x] -> m [y]`.

```
gatherViaMapM :: Monad m => [m a] -> m [a]
gatherViaMapM = undefined  -- YOUR CODE
```

Answer: [What are x and y in this use of mapM? What type must its function argument have?]

3.4. More generic monadic operations

Use only do/bind/return, ordinary application, and supplied arguments. `composeM` runs the first action-producing function and feeds its result to the second. `flattenM` removes one monadic layer. `liftTwoM` performs its two actions left-to-right and combines their results.

```
composeM :: Monad m => (a -> m b) -> (b -> m c) -> a -> m c
composeM = undefined  -- YOUR CODE

flattenM :: Monad m => m (m a) -> m a
flattenM = undefined  -- YOUR CODE

liftTwoM :: Monad m => (a -> b -> c) -> m a -> m b -> m c
liftTwoM = undefined  -- YOUR CODE
```

3.5. A limit of polymorphism

Is there a total implementation of `forall m a. Monad m => m a` for every `m` and `a`? Explain using both `Maybe` and `State ()`. Why does the existence of `Nothing` not settle the question?

Answer: [Reasoning about all monads and all result types.]

3.6. Changed implementation

Implement `repeatAction`: collect the results of performing one action `n` times, in order. For `n <= 0`, perform no actions. Use explicit recursion; no `replicateM`, `mapM`, `sequence`, or `traverse`.

```
repeatAction :: Monad m => Int -> m a -> m [a]
repeatAction = undefined  -- YOUR CODE
```

Answer: [Predict repeatAction 2 "ab", including order, before running it.]

Chapter 4

Evaluate monadic programs

Source: Sample Q4, p. 4; Monads lecture, do notation and List Monad.

Use this supplied function for Questions 4.1–4.2. Do not reimplement it here.

```
bundle :: Monad m => [m a] -> m [a]
bundle [] = return []
bundle (u:us) = do
  x <- u
  xs <- bundle us
  return (x:xs)
```

4.1. Values and types

Evaluate each expression and give its whole type. Leave function values such as `id` unevaluated when appropriate. All parts are independent.

(a) `bundle [Just 'a', Just 'c']`

Answer: [Result and type.]

(b) `bundle [Just not, Nothing]`

Answer: [Result and type.]

(c) `bundle [[] ::  [Bool]]`

Answer: [Result and type.]

(d) `bundle ([] ::  [[Bool]])`

Answer: [Result and type; distinguish this from part (c).]

(e) `bundle [[True,False], [True]]`

Answer: [All branches in order; whole type.]

(f) `bundle [[id,not], [const False]]`

Answer: [Function lists in order; then apply each returned function to True.]

4.2. Translate and trace

Rewrite the following without do notation, using binds and lambdas. Give each bound variable's type and the final result. Explain the effect of the middle line.

```
do x <- Just (6 :: Int)
   return (x+1)
   return (x*3)
```

```
-- YOUR TRANSLATION
```

Answer: [Types, result, explanation.]

4.3. Implement safe monadic computation

`safeQuot x y` returns `Nothing` for divisor zero and otherwise `Just (x `div` y)`. `divideChain` divides the starting value by each divisor left-to-right, failing on any zero; an empty divisor list succeeds with the starting value. Use explicit recursion for `divideChain`.

```
safeQuot :: Int -> Int -> Maybe Int
safeQuot = undefined  -- YOUR CODE

divideChain :: Int -> [Int] -> Maybe Int
divideChain = undefined  -- YOUR CODE
```

4.4. List choices and a generic fold

Implement `allPairs` using list do notation, with left choices outermost. Implement `foldActions` as a monadic left fold using recursion and binds; do not use library folds, `mapM`, or `sequence`.

```
allPairs :: [a] -> [b] -> [(a,b)]
allPairs = undefined  -- YOUR CODE

foldActions :: Monad m => (a -> b -> m a) -> a -> [b] -> m a
foldActions = undefined  -- YOUR CODE
```

4.5. Changed type question

Infer `\xs -> bundle (map Just xs)` and describe its behavior. Explain why its monad is more specific than the monad in `bundle`'s signature.

Answer: [Most general type and behavior.]

Chapter 5

State execution and implementation

Source: Sample Q1/Q2/Q4, pp. 1/2/4; State and StateMonad lectures.

Use this representation and the reference operations:

```
newtype State s a = S {runState :: s -> (a,s)}
```

In every trace, keep a column for the returned value and another for state after the step. Report `(result, finalState)` in that order.

5.1. Evaluate and infer

(a) `runState (get >>= \n -> put (n*2) >> get) (5 :: Int)`

Answer: [Step table, result, type.]

(b) `runState (put False >> get) True`

Answer: [Step table, result, type.]

(c) `runState (modify reverse >> get) "abc"`

Answer: [Step table, result, type.]

(d) `runState ((,) <$> get) 'q'`

Answer: [Returned function and final state; then apply the returned function to True.]

(e) Using `bundle` from chapter 4, evaluate the following with Boolean functions as state:

```
runState (bundle [get, put not >> get]) id
```

Answer: [Returned function list and final function; apply each to False.]

5.2. Construct State actions from types

`swapStore` returns the reversed pair while leaving the stored pair unchanged. `projectStore f x` returns `f currentState x`, also leaving state unchanged. Use only the abstract State interface; do not construct `S`.

```
swapStore :: State (a,b) (b,a)
swapStore = undefined  -- YOUR CODE

projectStore :: (a -> b -> c) -> b -> State a c
projectStore = undefined  -- YOUR CODE
```

Answer: [For each: stored-state type; returned-value type.]

5.3. Ordered updates

`fresh` returns the old integer counter and increments it once. `takeLabels n` collects `n` fresh labels; `n <= 0` leaves state unchanged. `pop` returns and removes the head of list state, or returns `Nothing` without changing empty state. No library traversal helpers.

```
fresh :: State Int Int
fresh = undefined  -- YOUR CODE

takeLabels :: Int -> State Int [Int]
takeLabels = undefined  -- YOUR CODE

pop :: State [a] (Maybe a)
pop = undefined  -- YOUR CODE
```

5.4. Label a tree

Use `fresh` to attach consecutive labels to leaves, left subtree before right, advancing the counter once per leaf. Do not construct `S` or call `runState` inside your implementation.

```
data Tree a = Leaf a | Branch (Tree a) (Tree a)
labelTree :: Tree a -> State Int (Tree (a,Int))
labelTree = undefined  -- YOUR CODE
```

5.5. Function-valued state

Return the currently stored Boolean function and replace the stored function by `old . not`. Use the abstract State interface.

```
snapshotFunction :: State (Bool -> Bool) (Bool -> Bool)
snapshotFunction = undefined  -- YOUR CODE
```

Answer: [How would you check both functions without Show or Eq instances for functions?]

5.6. Changed type question

Infer `\f x -> f <$> (put x >> get)`. Describe the symbolic result of running this action from a matching initial state `s0`.

Answer: [Most general type; result/state pair in terms of f, x, and s0.]

Chapter 6

Verification conditions

Source: Sample Q3, p. 3; DafnyVerificationConditions PDF, physical pp. 3–30.

These are written partial-correctness questions, not requests to run a verifier. State the induced formulas even if a proposed invariant is invalid. Distinguish deriving an obligation from proving it. Ignore termination and runtime-safety obligations in this practice set unless explicitly asked.

6.1. Assignment and sequence

Compute each weakest precondition. Show substitution, working backward.

(a) `x := x + 3;` with postcondition $x \geq 10$.

Answer: [Substitution and simplified precondition.]

(b) `x := y + 2; y := x - 1;` with postcondition $x > y \wedge y \geq 0$.

Answer: [Precondition before each assignment; simplified result.]

(c) `x := x + y; y := x - y;` with postcondition $y = 7$.

Answer: [Show both substitutions. Assignments are sequential.]

6.2. Conditional

Compute a weakest precondition for this code with postcondition $z \geq 0$. First write the guarded implications for both branches; then simplify if useful.

```
if x < y {
  z := y - x;
} else {
  z := x - y;
}
```

Answer: [Branch preconditions; guarded conjunction; simplification.]

6.3. Derive all three loop obligations

```
method TwicePlus(n:int) returns (t:int)
  requires n > 0
  ensures t == 2*n + 3
{
  t := 0;
  var i := n;
  while i > 0
  {
    t := t + 2;
    i := i - 1;
  }
  t := t + 3;
}
```

For **each** proposed invariant below, list (1) initialization after substituting the initialization, (2) preservation after substituting the body, and (3) exit after accounting for the final assignment. Then state which obligations hold. For a failed implication, give a concrete counterexample satisfying its antecedent. A counterexample need not be a reachable loop state.

(a) $i \geq 0 \wedge t + 2i = 2n$.

Answer: Initialization: [ ]. Preservation: [ ]. Exit: [ ]. Validity/counterexamples: [ ].

(b) $i \geq 0 \wedge t = 2i$.

Answer: Initialization: [ ]. Preservation: [ ]. Exit: [ ]. Validity/counterexamples: [ ].

(c) $i \geq 0$.

Answer: Initialization: [ ]. Preservation: [ ]. Exit: [ ]. Validity/counterexamples: [ ].

6.4. Repair and transfer

Now change the body update to `t := t + 3` and the postcondition to $t = 3n + 3$, retaining the final `t := t + 3`. Propose an invariant and derive the three obligations. Include bounds you need at loop exit.

Answer: Invariant: [ ]. Initialization: [ ]. Preservation: [ ]. Exit: [ ].

Chapter 7

Laziness and summary-producing recursion

Source: Sample Q5, p. 5 (helper structure); Dynamic lecture (laziness context).

Use this supplied helper. It is also available in the optional Haskell file.

```
spreadMax :: [Int] -> Int -> (Int,[Int])
spreadMax [] seed = (seed,[])
spreadMax [x] seed = (x,[seed])
spreadMax (x:xs) seed = (max x rest, seed:out)
  where (rest,out) = spreadMax xs seed
```

7.1. Evaluate the helper

(a) `spreadMax [] 20`

Answer: [Result.]

(b) `spreadMax [4] 20`

Answer: [Result.]

(c) `spreadMax [4,1,7] 20`

Answer: [Recursive intermediate pairs and final result.]

(d) `spreadMax [-8,-3,-6] 0`

Answer: [Result; does seed participate in the summary for nonempty input?]

7.2. Explain the two outputs

For an arbitrary finite nonempty input, describe the first and second components. Which input determines the length of the output list? Which component can be computed without forcing the numeric seed? Justify from the equations rather than assuming that every argument is evaluated first.

Answer: [Component meanings and dependency explanation.]

7.3. Implement with two helper calls

Implement a function replacing every element by the maximum of the original list. Return `[]` on `[]`. Examples: `[4,1,7] -> [7,7,7]` and `[-8,-3] -> [-3,-3]`. Use exactly two calls to `spreadMax` for nonempty input and pair projections or let bindings. No explicit recursion, `maximum`, `map`, `replicate`, folds, or other recursive list helpers.

```
toMaximumTwoPass :: [Int] -> [Int]
toMaximumTwoPass = undefined  -- YOUR CODE
```

7.4. One-call challenge

Implement the same behavior with exactly one call to `spreadMax` for nonempty input, using a recursive let/where binding. The other restrictions remain. Handle the empty list explicitly.

```
toMaximumOneCall :: [Int] -> [Int]
toMaximumOneCall = undefined  -- YOUR CODE
```

Answer: [Draw or describe the dependencies. Why can the summary be determined?]

7.5. Changed independent helper

Use the helper below to return a list with each position replaced by the sum of all input elements. Use one helper call, no other recursive list function, and handle the empty list explicitly.

```
spreadSum :: [Int] -> Int -> (Int,[Int])
spreadSum [] seed = (0,[])
spreadSum (x:xs) seed = (x+rest,seed:out)
  where (rest,out) = spreadSum xs seed
```

```
toTotalOneCall :: [Int] -> [Int]
toTotalOneCall = undefined  -- YOUR CODE
```

Answer: [Explain why the changed base case matters. Predict output for [2,-1,4].]

7.6. Productivity is a dependency question

Compare `let x = x + 1 in x` with `let xs = 1 :  xs in take 3 xs`. Explain which finite observation can be produced and why. Do not conclude that every circular binding is safe.

Answer: [Dependency reasoning and result where one exists.]

Chapter 8

Mixed review: five sample-style questions

This is a fresh practice set modeled on the sample's Q1–Q5. It is not a mock with a verified current format, timing, or grading scheme. First attempt without earlier solutions, GHCi, or the reference. Record any assistance afterward.

Question 1. Most general types or ill-typed

Use the State interface from the reference. All parts are independent. Include constraints and briefly justify any type error.

(a) `\xs -> [show x | x <- xs, even x]`

Answer: [Answer.]

(b) `get >>= \xs -> return (length xs)`

Answer: [Answer.]

(c) `modify (map Just)`

Answer: [Answer.]

(d) `put True >> return 'k'`

Answer: [Answer.]

(e) `(:)  <$> get`

Answer: [Answer.]

(f) `\f mx -> fmap f mx >>= \y -> return (y,y)`

Answer: [Answer.]

Question 2. Construct expressions from types

Give useful total implementations of the described behavior. Use the abstract State interface. For `collectEither`, use explicit recursion, not traversal helpers. It returns all Right values or the first Left error, left-to-right. `applyAfterRead f` applies `f` to current state without changing it. `rotateStore` changes triple state from `(x,y,z)` to `(z,x,y)` and returns the old triple. `liftThree` performs three actions left-to-right and combines their results; use do/bind/return, not lifting helpers.

```
collectEither :: [Either e a] -> Either e [a]
collectEither = undefined  -- YOUR CODE

applyAfterRead :: (s -> a) -> State s a
applyAfterRead = undefined  -- YOUR CODE

rotateStore :: State (a,a,a) (a,a,a)
rotateStore = undefined  -- YOUR CODE

liftThree :: Monad m => (a -> b -> c -> d) -> m a -> m b -> m c -> m d
liftThree = undefined  -- YOUR CODE
```

Can a total function of type `Monad m => m a -> a` work for all monads and all result types? Explain with a counterexample specialization.

Answer: [Answer and reasoning.]

Question 3. Induced verification conditions

Use the mathematical function $\text{Squares}(n) = 0$ for $n \leq 0$, and $\text{Squares}(n) = n^2 + \text{Squares}(n-1)$ otherwise.

```
method SumSquares(n:int) returns (t:int)
  requires n > 0
  ensures t == Squares(n) + 4
{
  t := 0;
  var i := n;
  while i > 0 {
    t := t + i*i;
    i := i - 1;
  }
  t := t + 4;
}
```

For each invariant, list initialization, preservation, and exit obligations, including the suffix after the loop. Derive formulas even for an invalid invariant. This question asks for partial correctness; termination is outside this set.

(a) $i \geq 0 \wedge t + \text{Squares}(i) = \text{Squares}(n)$.

Answer: Initialization: [ ]. Preservation: [ ]. Exit: [ ].

(b) $i \geq 0 \wedge t = \text{Squares}(n - i)$.

Answer: Initialization: [ ]. Preservation: [ ]. Exit: [ ].

(c) $t = \text{Squares}(i)$.

Answer: Initialization: [ ]. Preservation: [ ]. Exit: [ ].

Question 4. Trace and rewrite a generic program

```
pack :: Monad m => [m (a -> b)] -> m [a -> b]
pack [] = return []
pack (u:us) = do
  f <- u
  fs <- pack us
  return (f:fs)
```

Evaluate each expression, leaving functions as `id`, `not`, etc. where needed. For (e), both initial and stored functions are Boolean functions.

(a) `pack [Just not]`

Answer: [Answer.]

(b) `pack [Just not, Nothing]`

Answer: [Answer.]

(c) `pack [[] ::  [Bool -> Bool]]`

Answer: [Answer.]

(d) `pack [[not,id], [const True, const False]]`

Answer: [Answer in enumeration order.]

(e) `runState (pack [get, modify (.  not) >> get]) id`

Answer: [Returned functions and final stored function; apply each to True to check.]

Reimplement `pack` using exactly one call to `mapM`, without recursion.

```
packViaMapM :: Monad m => [m (a -> b)] -> m [a -> b]
packViaMapM = undefined  -- YOUR CODE
```

Question 5. Lazy summary and reconstruction

```
scatterMin :: [Int] -> Int -> (Int,[Int])
scatterMin [] seed = (seed,[])
scatterMin [x] seed = (x,[seed])
scatterMin (x:xs) seed = (min x rest,seed:out)
  where (rest,out) = scatterMin xs seed
```

Evaluate:

(a) `scatterMin [] 11`

Answer: [Answer.]

(b) `scatterMin [8] 11`

Answer: [Answer.]

(c) `scatterMin [8,3,6] 11`

Answer: [Answer.]

(d) `scatterMin [8,3,6] (-20)`

Answer: [Answer.]

Implement replacement of every element by the list minimum, returning an empty list on empty input. Use exactly two calls to `scatterMin` for nonempty input. Do not use explicit recursion, folds, `minimum`, `map`, `replicate`, or other recursive list functions. Then give a one-call version under the same restrictions using a lazy recursive binding.

```
toMinimumTwoPass :: [Int] -> [Int]
toMinimumTwoPass = undefined  -- YOUR CODE

toMinimumOneCall :: [Int] -> [Int]
toMinimumOneCall = undefined  -- YOUR CODE
```

Answer: [Explain why the one-call binding is productive on finite nonempty inputs.]

After your attempt

Record which questions were independent, hinted, or completed after an explanation. Choose one type error to revisit with a changed example.

Answer: [Evidence and next review target; no predicted instructor grade.]

Chapter 9

Lecture extensions: coverage unconfirmed

These topics occur in your lectures but are not explicitly asked on the 2025 sample. Attempt after the core sets, or when an announcement confirms scope. These are supplementary questions, not predictions of exam questions.

9.1. Transformer runner types

Use these supplied representations:

```
runStateT  :: StateT s m a -> s -> m (a,s)
runExceptT :: ExceptT e m a -> m (Either e a)
```

Expand each into a function type with no remaining transformer wrappers: (a) `StateT Int (Either String) Bool`; (b) `ExceptT String (State Int) Bool`, after running both layers. Which ordering can return a final state together with an error? Justify from the expanded result shapes.

Answer: [Expanded types and reasoning.]

9.2. Immutable arrays and dynamic programming

Using the Dynamic lecture's array-based edit-distance definition, identify the base cells and the three predecessor cells for a mismatch. Explain why calling the naive recursive function rather than looking up the array in the recursive cases would lose sharing of subproblems.

Answer: [Recurrence, base cases, sharing explanation. Source: Dynamic.hs, Recursion over indices and self-referential array.]

9.3. GADT evaluator

Use this indexed expression language. Implement the evaluator directly below.

```
data Expr a where
  Number :: Int -> Expr Int
  Truth  :: Bool -> Expr Bool
  Plus   :: Expr Int -> Expr Int -> Expr Int
  If     :: Expr Bool -> Expr a -> Expr a -> Expr a
```

```
evalExpr :: Expr a -> a
evalExpr = undefined  -- YOUR CODE
```

Does the following expression type-check? Explain using the constructor types.

```
Plus (Truth True) (Number 2)
```

Source: GADTs.hs, A Typed Expression Evaluator.

Answer: [Answer and type conflict, if any.]

9.4. Agda and propositions as types

Use the tutorial's correspondence between implication and functions, conjunction and products, disjunction and sums, and falsity and an empty type. State the type representing $(P \wedge Q) \Rightarrow Q$, then implement its proof. Use the tutorial's product constructor/notation or define an equivalent record. Explain why negation of P corresponds to $P \to \bot$, rather than to $\bot$ alone.

```
-- YOUR AGDA TYPE AND PROOF
```

Answer: [Negation explanation. Source: Tutorial.agda, Curry-Howard correspondence.]

Appendix A

Study reference

This is a supplementary reference, not an official exam appendix. Try mixed review before opening it. These are the signatures/semantics used in our files.

```
id      :: a -> a
const   :: a -> b -> a
(.)     :: (b -> c) -> (a -> b) -> a -> c
($)     :: (a -> b) -> a -> b
fst     :: (a,b) -> a
snd     :: (a,b) -> b
(,)     :: a -> b -> (a,b)
(:)     :: a -> [a] -> [a]
Just    :: a -> Maybe a
Nothing :: Maybe a
map     :: (a -> b) -> [a] -> [b]
filter  :: (a -> Bool) -> [a] -> [a]
concat  :: [[a]] -> [a]
length  :: [a] -> Int
null    :: [a] -> Bool
head    :: [a] -> a       -- partial on []
tail    :: [a] -> [a]     -- partial on []
even    :: Integral a => a -> Bool
show    :: Show a => a -> String
(+)     :: Num a => a -> a -> a
(==)    :: Eq a => a -> a -> Bool
max     :: Ord a => a -> a -> a
fmap, (<$>) :: Functor f => (a -> b) -> f a -> f b
pure    :: Applicative f => a -> f a
(<*>)   :: Applicative f => f (a -> b) -> f a -> f b
return  :: Monad m => a -> m a
(>>=)   :: Monad m => m a -> (a -> m b) -> m b
(>>)    :: Monad m => m a -> m b -> m b
mapM    :: Monad m => (a -> m b) -> [a] -> m [b]
sequence :: Monad m => [m a] -> m [a]
```

```
newtype State s a = S {runState :: s -> (a,s)}
get       :: State s s
put       :: s -> State s ()
modify    :: (s -> s) -> State s ()
evalState :: State s a -> s -> a
execState :: State s a -> s -> s
```

State and verification reminders

`runState` returns `(result, finalState)`. Function results cannot normally be printed; apply them to an argument before comparing output. `return` constructs an action; it does not exit a `do` block. `Maybe` bind propagates `Nothing`; list bind concatenates each branch's results in order. State bind passes the updated state to the next action.

For an invariant I, guard B, body S, initialization A, suffix T, method precondition P, and postcondition Q, our partial-correctness obligations are

$$P \Rightarrow wp(A, I), \qquad I \wedge B \Rightarrow wp(S, I), \qquad I \wedge \neg B \Rightarrow wp(T, Q).$$

Assignment substitutes in the postcondition; sequence works backward. For conditionals, use guarded implications for both branches. These obligations do not by themselves prove termination or all runtime-safety properties.